

Machine Learning Operations (MLOps)

Foundations of Production-Ready ML

Prof. Surya Prakash
IIT Indore

Let's first understand DevOps

- Software development = Development + Deployment
- Development
 - It refers to the process of creating and building software applications.
 - It is the part where the Development Team writes, tests, and prepares the code before it is deployed.
- Deployment
 - It refers to the process of making a software application available for use in a live environment.
 - It is what happens after development is complete.

Let's first understand DevOps

- DevOps is a set of practices, tools, and a culture that brings Development (Dev) and Operations (Ops) teams together to build, test, deploy, and operate software faster and more reliably.
- In simple terms
 - DevOps = faster delivery + better quality + stable systems
- What DevOps focuses on
 - Collaboration between developers and operations
 - Automation of build, test, and deployment
 - Continuous delivery of software (no downtime)
 - Monitoring and feedback in production

Let's first understand DevOps

- Key DevOps practices
 - CI/CD – Continuous Integration & Continuous Deployment
 - Version control – Git
 - Containerization – Docker
 - **Docker** = The container builder/runner (creates the boxes)
 - **Kubernetes** = The orchestrator (manages many boxes across multiple machines)
 - Orchestration – Kubernetes (Kubernetes is an open source and the most widely used tool for container orchestration platform.)
 - Orchestration (managing) refers to the automated management, coordination, and scaling of containerized applications.
 - Monitoring and logging – Prometheus, Grafana, ELK
 - **Prometheus**: monitors model latency, throughput, resource usage
 - **Grafana**: visualizes model performance and system health
 - **ELK**: stores inference logs, errors, and prediction metadata
 - Infrastructure as Code (IaC)
 - practice of managing and provisioning IT infrastructure through machine-readable code instead of manual processes.
 - Humans write the code and machines read and execute it to create infrastructure
- Why DevOps is important
 - Faster releases
 - Fewer deployment failures
 - Faster recovery from failures
 - Consistent environments (Dev / Test / Prod)

Examples of machine-readable IaC code:
Terraform (HCL-HashiCorp Configuration Language)

```
resource "aws_instance" "web" {  
  ami          = "ami-0abcd1234"  
  instance_type = "t2.micro"  
}
```

→ A tool reads this file and **creates a virtual machine automatically**.

→ **Terraform** is an **IaC** tool used to **provision and manage cloud and on-premise infrastructure** using **machine-readable configuration files** written in **HCL**.

Example: Deploying a Web Application with DevOps

- Scenario:
 - A team wants to release a new feature on their e-commerce website.
- Traditional Approach (Without DevOps):
 - Developer writes code → sends to operations team → manually deploys → sometimes causes downtime → slow feedback

Example: Deploying a Web Application with DevOps

- DevOps Approach:
 - Code Commit:
 - Developer pushes code to Git (version control)
 - Continuous Integration (CI):
 - Automated tests run to check code quality
 - Build & Containerize:
 - Application is packaged into a Docker container
 - Continuous Deployment (CD):
 - Pipeline automatically deploys the container to **staging environment**
 - Staging environment: It is a pre-production environment that mimics the production environment as closely as possible.
 - Monitoring & Feedback:
 - Tools like Prometheus/Grafana check performance, errors, and user impact
 - Production Release:
 - Once tests pass, the feature is deployed to production, live for users
 - Rollback (if needed):
 - If a problem occurs, deployment can be quickly reverted

Example: Deploying a Web Application with DevOps

- **Benefits in this example:**
 - Faster feature release
 - Fewer errors in production
 - Consistent environment from development → production
 - Continuous feedback improves future releases

Development of ML based Software Products

- Two ways to develop machine learning based software products
 - Example: A recommendation system in an e-commerce app.
- (1) Machine Learning-Based Software Development
 - This is the process of creating software applications that include machine learning models.
 - It involves:
 - Data collection & preprocessing
 - Model training & evaluation
 - Integration of the model into an application
 - Testing & deployment
- Machine Learning-Based Software Development often involves manual steps like coding, training models, testing, and deploying them individually.

Development of ML based Software Products

- (2) MLOps (Machine Learning Operations)
 - MLOps is like DevOps for ML — it ensures that ML models are production-ready, scalable, reliable, and maintainable.
 - MLOps automates and streamlines many of the steps like retraining, deployment, monitoring, but humans are still involved in decisions, designing pipelines, and model updates.
 - It focuses on the lifecycle management of ML models, including:
 - Versioning: Keeping track of data, code, and model versions.
 - Automation: Automating training, testing, and deployment pipelines.
 - Monitoring: Tracking model performance over time.
 - Collaboration: Making it easier for data scientists, ML engineers, and software developers to work together.

MLOps

- MLOps (Machine Learning Operations)
 - It is related to machine learning-based software development and deployment (with automating steps like retraining, deployment, monitoring)
 - MLOps defines a set of practices that combines Machine Learning, Software Engineering, and DevOps to manage the entire lifecycle of ML systems
 - It deals with data and model development to deployment, monitoring, and governance.
- Purpose:
 - To enable reliable development, deployment, monitoring, and governance of machine learning systems in production.

MLOps

- **Core goals:**
 - Reproducibility (same code + data → same results)
 - Automation (CI/CD for ML)
 - Scalability (serving and retraining)
 - Reliability (testing, monitoring, rollback)
 - Responsibility (Fairness, Bias Checks, Transparency)

Reproducibility (Same code + data → same results)

- Reproducibility means that an ML experiment can be recreated exactly at any time.
- This means that using the **same source code**, the **same dataset**, and the **same parameters** will produce the same model and identical results.
- Why does it matter?
 - Ensures scientific validity
 - Enables comparison of experiments
 - Helps in debugging
- How does MLOps support it?
 - Git for code versioning
 - DVC (Data Version Control) for data, and model, and ML pipelines versioning
 - Configuration tracking (recording, versioning, and managing all parameters and settings)

Automation (CI/CD for ML)

- Automation refers to automatically running ML workflows **without** manual intervention.
- It means using CI/CD pipelines to automatically **build**, **test**, **validate**, and **deploy** machine learning models whenever changes occur in **code**, **data**, or **configuration**.
 - **CI: Continuous Integration** (integrating and validating changes frequently)
 - **CD: Continuous Deployment/Delivery** (about delivering validated changes to an environment)

Automation (CI/CD for ML)

- CI (Continuous Integration) in ML
 - CI automatically runs when new code or data is committed:
 - Code linting and unit tests
 - LINTING: automatically analyzing code to detect syntax errors, style violations, potential bugs, bad coding practices) Data validation checks
 - Model training sanity checks
 - quick, automated validations run during or immediately after training to ensure the model is learning correctly and behaving as expected
 - RAG pipeline validation (retrieval + generation)
 - the process of verifying that both components of a RAG system, the retriever and the generator (LLM), are working correctly and together producing grounded, relevant, and reliable outputs.
- Purpose:
 - To catch errors early and ensure only valid models move forward.

Automation (CI/CD for ML)

- CD (Continuous Deployment/Delivery) in ML
 - CD automates what happens after CI passes:
 - Packaging the model
 - Deploying it as an API or service
 - Updating model versions
 - Rolling back if issues occur
- Purpose:
 - To deliver models to production quickly and safely.

Automation (CI/CD for ML)

- It includes:
 - Automated testing of code and models
 - Automatic training and validation
 - Automated deployment pipelines
- Why is it important?
 - ML systems change due to data, not just code
 - Manual deployment is error-prone
 - Automation ensures consistency and speed
 - Speeds up development
 - Ensures consistency across environments
- Example:
 - When new code is pushed, CI automatically tests the model before deployment.

Scalability (Serving and Retraining)

- Scalability means the ML system can handle increased users, data, and workload.
- Two aspects:
 - Serving scalability: Handling many inference requests
 - Training scalability: Retraining models on larger or new datasets
- Why is it important?
 - Real-world ML systems grow over time
 - Models must adapt to new data
- MLOps role:
 - Containerization and orchestration
 - Automated retraining pipelines

Reliability (Testing, Monitoring, Rollback)

- **Reliability** ensures that ML systems work **correctly and consistently** in production.
- Key practices:
 - Unit and integration testing
 - Continuous monitoring of performance
 - Rollback to previous stable models if issues occur
- Why is it important?
 - Prevents system failures
 - Maintains user trust
 - Enables fast recovery from errors

Responsibility (Fairness, Bias Checks, Transparency)

- Responsibility focuses on building ethical and trustworthy AI systems.
- Includes:
 - Detecting and reducing bias in data and predictions
 - Ensuring fairness across user groups
 - Providing transparent and explainable outputs
- Why is it important?
 - Prevents discrimination
 - Ensures compliance with ethical standards
 - Builds trust in AI systems
- For example, in RAG systems:
 - Showing source documents
 - Monitoring harmful or biased outputs

Key components of MLOps

- Versioning: Git (code), DVC (data/models)
 - code: it is the instructions written by developers.
 - Model: it is the result produced by running the code on data.
- CI/CD:
 - automated tests, validation, deployment
- Serving:
 - APIs (e.g., FastAPI)
- Monitoring:
 - performance, drift, bias
- Governance:
 - fairness & responsible AI

ML + RAG System

- A Retrieval-Augmented Generation (RAG) system combines:
 - Information retrieval from documents or databases
 - Prompt augmentation using retrieved context
 - Text generation using Large Language Models (LLMs)
- MLOps is critical because RAG systems depend on both models and continuously evolving data.

ML + RAG System

- A ML + RAG team typically builds an end-to-end intelligent application, not just a model.
- They build:
 - Data ingestion & preprocessing pipelines
 - Data ingestion: collecting, importing, and loading raw data
 - Embedding generation and vector database (for RAG)
 - Retrieval + LLM-based generation logic
 - Model training / fine-tuning (if needed)
 - MLOps pipelines for deployment, automation, monitoring, and updates

ML + RAG System

- Example
 - a university chatbot where the ML + RAG team builds a system that retrieves rules, notices, and syllabi from documents and uses an LLM to generate accurate answers, with MLOps pipelines to automatically update data, redeploy models, and monitor performance.

Quiz

1. What is the primary difference between traditional Machine Learning-Based Software Development and MLOps?

- A) MLOps uses Python while traditional ML uses R
- B) MLOps automates and streamlines steps like retraining, deployment, and monitoring
- C) Traditional ML is faster than MLOps
- D) MLOps doesn't require data preprocessing

Answer: B

2. In the context of Infrastructure as Code (IaC), who "reads" the machine-readable code?

- A) Developers only
- B) Operations team only
- C) Machines/Tools (like Terraform)
- D) End users

Answer: C

3. Which of the following is NOT a core goal of MLOps?

- A) Reproducibility
- B) Automation
- C) Cost Reduction
- D) Responsibility

Answer: C

Quiz

4. What does DVC (Data Version Control) track in MLOps?

- A) Only datasets
- B) Only models
- C) Data, models, and ML pipelines
- D) Only code versions

Answer: C

5. In a DevOps workflow, what is a "staging environment"?

- A) The final production environment
- B) A pre-production environment that mimics production as closely as possible
- C) The developer's local machine
- D) A backup server

Answer: B

6. What does "linting" mean in the context of CI (Continuous Integration)?

- A) Compiling code into machine language
- B) Automatically analyzing code to detect syntax errors, style violations, and potential bugs
- C) Creating documentation automatically
- D) Encrypting sensitive code

Answer: B

Quiz

7. Which tool of following is used for container orchestration in the document?

- A) Docker
- B) Prometheus
- C) Kubernetes
- D) Terraform

Answer: C

8. What does reproducibility in MLOps ensure?

- A) Models run faster in production
- B) Same code + data → same results
- C) Less storage is required
- D) Models are smaller in size

Answer: B

9. What is the purpose of "model training sanity checks" in CI for ML?

- A) To make models train faster
- B) To reduce model size
- C) Quick validations to ensure the model is learning correctly and behaving as expected
- D) To eliminate the need for testing

Answer: C

Quiz

10. In the DevOps, what happens if a problem occurs after production release?

- A) The entire system shuts down
- B) Manual debugging is required before any action
- C) Deployment can be quickly reverted (rollback)
- D) Users must wait for the next release

Answer: C

11. What does RAG pipeline validation verify?

- A) Only the retrieval component
- B) Only the generation component
- C) That both retriever and generator are working correctly together
- D) The database connection only

Answer: C

12. What does Prometheus monitor in the context of ML systems?

- A) Only code quality
- B) Model latency, throughput, and resource usage
- C) Only database performance
- D) User authentication

Answer: B

Quiz

13. In containerization, what does a container package together?

- A) Only the application code
- B) Only the dependencies
- C) Application code, dependencies, libraries, and runtime environment
- D) Only the database

Answer: C

14. What is the correct sequence of steps in the DevOps approach for deploying a web application feature?

- A) Code Commit → Build & Containerize → CI → CD → Monitoring → Production Release
- B) Code Commit → Continuous Integration → Build & Containerize → Continuous Deployment → Monitoring & Feedback → Production Release
- C) Build & Containerize → Code Commit → CI → Production Release → Monitoring
- D) Code Commit → Production Release → CI → CD → Monitoring

Answer: B

15. In the context of MLOps, what does "configuration tracking" involve?

- A) Only tracking which developers made changes
- B) Recording, versioning, and managing all parameters and settings
- C) Monitoring server configurations only
- D) Tracking user preferences

Answer: B

Thank You